DASH-01: Dashboard Fundamentals

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 1 of 7 | **Created:** March 2026 | **Last Updated:** 07/30/2026

Overview

Dashboards are the primary visualization layer in Dynatrace, turning raw observability data into actionable insights. This notebook covers the distinction between dashboards and notebooks, when to use each, the architecture of Dynatrace dashboards (tiles, sections, variables), and core design principles that make dashboards effective. Whether you are building your first dashboard or refining an existing library, these fundamentals provide the foundation for every notebook in this series.

Sprint 1.344 (July 2026): Dashboards Failing Validation No Longer Load

> **Forthcoming/rolling out (SaaS 1.344): a dashboard that fails validation will no longer load.** SaaS 1.344 was published 07/27/2026 with a **staged tenant rollout from 07/29/2026** — verify whether it has reached your tenant before relying on the new behavior. Before 1.344, a dashboard whose payload failed validation still loaded and surfaced validation warnings. Once 1.344 reaches your tenant, it does not load at all.
>
> Dynatrace names dashboards **created externally via API or by AI tooling** as the most affected population. If you build dashboards only in the UI, this changes little — the UI emits shapes it can render. If you deploy dashboards as code (Documents API, Monaco, Terraform) or generate them with a script or an assistant, treat the warning state as a grace period rather than a supported state: deploy to a non-production tenant, open the dashboard in the Dashboards app, and confirm zero validation warnings before merging. **DASH-07 §5** carries the full gate, including why a `2xx` from the Documents API is not render confirmation.

Sprint 1.337 (April 2026): New Dashboard Building Blocks

Sprint 1.337 introduced data shapes that make several dashboard patterns simpler:

1. **OneAgent primary fields/tags as top-level fields** on all signals (Latest Dynatrace). Filters and `by:` groupings can dispatch on `dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`, and customer-defined `primary_tags.<key>` directly — no more `parse(content, ...)` in dashboard tile queries. Particularly impactful for executive dashboards (DASH-03) that need cost-by-business-unit views.
2. **Smartscape Ownership integration** — entities now carry ownership (`ownership.team`, `ownership.oncall`) as queryable attributes. Dashboards can split metrics or surface alerts by owning team without maintaining side-tables. See:

`dql smartscapeNodes "SERVICE" | fieldsAdd team = getNodeField(smartscape.id, "ownership.team") | summarize service_count = count(), by:{team} | sort service_count desc`

3. **OTel** **`service.name`** **enrichment** + new **`dt.service.name`** field — splitting service-level dashboards by OTel-canonical name now works without joining through entity enrichment.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`storage:logs:read`, `storage:metrics:read`, `storage:events:read`
Access	Dashboard creation privileges (`document:documents:write`)
Data	At least 1 hour of ingested logs and metrics

1. Dashboards vs Notebooks

Dynatrace provides two primary visualization tools, each designed for different use cases.

Feature	Dashboard	Notebook
Purpose	Ongoing monitoring and reporting	Ad-hoc exploration and analysis
Layout	Grid-based tile arrangement	Sequential, document-style cells
Audience	Teams, stakeholders, wall screens	Individual analysts and engineers
Interactivity	Variables, filters, drill-down links	Inline DQL editing, iterative queries
Refresh	Auto-refresh (configurable interval)	Manual execution per cell
Sharing	Link sharing, embed, scheduled reports	Share as document or export
Persistence	Saved as Dynatrace documents	Saved as Dynatrace documents or `.ipynb`

When to Use a Dashboard

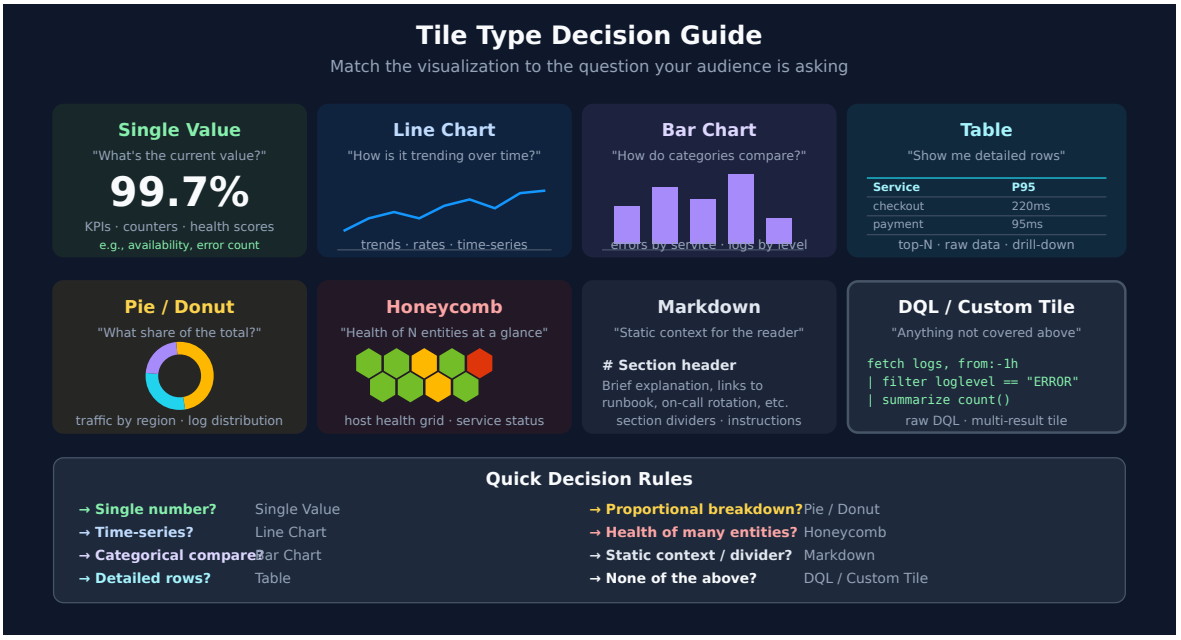
- **Continuous monitoring** — service health, SLA tracking, infrastructure overview
- **Team visibility** — wall screens, shared views across roles
- **Stakeholder reporting** — executive summaries, weekly status

- **Alerting context** — a landing page linked from alert notifications

When to Use a Notebook

- **Incident investigation** — iterating on queries to find root cause
- **Data exploration** — discovering patterns in new data sources
- **Knowledge sharing** — documented analysis with explanatory text
- **Training** — step-by-step DQL tutorials (like this series)

2. Dashboard Architecture



A Dynatrace dashboard is composed of three core building blocks:

Tiles

Tiles are the individual visualization units. Each tile contains a single DQL query (or metric expression) and renders as a specific chart type.

Tile Type	Best For	Example
Single value	KPIs, counters, health scores	Error count, availability %
Line chart	Trends over time	CPU usage, request rate
Bar chart	Categorical comparisons	Errors by service, logs by level

Tile Type	Best For	Example
Table	Detailed data, top-N lists	Top 10 slowest endpoints
Pie / Donut	Proportional breakdowns	Traffic by region
Honeycomb	Entity health at a glance	Host health grid
Markdown	Static text, instructions	Section headers, links

Sections

Sections group related tiles visually. Use sections to organize dashboards by theme — for example, a "Service Health" section alongside an "Infrastructure" section. Sections can be collapsed and rearranged.

Variables

Variables make dashboards dynamic. A single dashboard template can serve multiple teams by letting users select entities, environments, or time ranges from dropdown controls. Variables are covered in depth in **DASH-06**.

Release Radar (April 2026): New Tile Types and Authoring Controls

Recent additions to the dashboard authoring surface:

Addition	What it does	Where it helps
Treemap visualization	New tile type for hierarchical data — area encodes magnitude so dominant categories pop out (e.g. requests per service, grouped by Kubernetes namespace)	Spotting the big contributors in nested categories where a bar chart would need many rows
Row marker coloring (tables)	Color-coded row markers visually group related rows while keeping the table readable	Operations tables (DASH-04) where rows belong to tiers, teams, or severities

Addition	What it does	Where it helps
Centralized tile indicator controls	Show/hide warnings, descriptions, and custom timeframes across *all* tiles from one dashboard-level control	Cleaning up a dense dashboard for a wall screen vs. an editing session
Dashboard variables for dynamic coloring	Variables can drive coloring and threshold conditions, so visual rules stay in sync with the selected environment/team	Reusable templates — see **DASH-06**
Direct image upload in Markdown	Built-in image library: upload images directly inside Markdown tiles across Dashboards, Notebooks, and Launcher	Embedding logos, legends, or architecture snippets without an external host

Add **Treemap** to the tile-type vocabulary alongside the chart types in the table above. It is the right choice when you want *share-of-total within a hierarchy* at a glance, rather than precise per-category values.

3. Design Principles

Effective dashboards follow consistent design principles regardless of the audience.

Clarity

- **One purpose per dashboard** — avoid "everything" dashboards that try to serve every audience
- **Descriptive tile titles** — a tile titled "Error Rate (%) — Checkout Service" is immediately understandable; "Query 1" is not
- **Consistent time ranges** — all tiles should use the dashboard time selector unless there is a specific reason for a fixed range

Audience Awareness

- **Executives** need 3-5 high-level KPIs, not 30 detailed charts
- **Operations** teams need real-time status and alert context
- **Engineers** need drill-down capability and raw data access

Design for your audience first, then consider the data.

Actionability

Every tile should answer a question or trigger an action:

Tile Shows	Action
Error rate above threshold	Investigate in notebook or traces
SLA breach trending	Escalate to engineering
Deployment marker + latency spike	Correlate with release
Host CPU saturated	Scale or investigate process

If a tile does not lead to a decision, consider removing it.

Layout Best Practices

- **Top-left anchor** — place the most critical KPI in the top-left (first thing the eye hits)
- **Z-pattern reading** — arrange tiles so the dashboard reads left-to-right, top-to-bottom
- **Group related tiles** — use sections to cluster related metrics
- **Limit tile count** — aim for 8-12 tiles per dashboard; more than 15 creates cognitive overload

4. Dashboard Creation Workflow

Follow this structured approach when building a new dashboard:

Step 1: Define the Audience and Purpose

Answer these questions before opening the dashboard editor:

- **Who** will view this dashboard? (role, technical level)
- **What** decisions will it support?
- **When** will it be viewed? (real-time wall screen, weekly review, incident response)
- **How often** should data refresh?

Step 2: Identify Key Metrics

Select 5-10 metrics that directly answer the audience's questions. Validate each query in a notebook first.

Step 3: Prototype in a Notebook

Write and validate all DQL queries in a notebook before building the dashboard. This lets you iterate quickly without fighting the tile editor.

Step 4: Build the Dashboard

Transfer validated queries to dashboard tiles. Arrange tiles in sections following the layout principles above.

Step 5: Add Variables and Filters

Make the dashboard reusable by adding entity and time range variables.

Step 6: Share and Iterate

Share with the target audience, collect feedback, and refine. A dashboard is never "done" — it evolves with the team's needs.

5. Your First Dashboard Query

Before building a dashboard, you need validated DQL queries. Let's start with a few foundational queries that appear on almost every dashboard.

Total Log Volume by Level

This query provides a quick overview of log health — useful as a bar chart tile.

```
// Log volume breakdown by severity level
fetch logs, from:-1h
| summarize log_count = count(), by:{loglevel}
| sort log_count desc
```

Active Problem Count

A single-value tile showing how many detected problems are currently active — the most common executive KPI.

```
// Count of currently active detected problems
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| summarize active_problems = countDistinct(event.id)
```

Error Log Trend Over Time

A line chart showing error frequency — ideal for spotting spikes and correlating with deployments.

```
// Error log trend over the last 6 hours
fetch logs, from:-6h
| filter loglevel == "ERROR"
| makeTimeseries error_count = count(), interval:10m
```

Host CPU Overview

A timeseries query for a line chart showing CPU usage across hosts.

```
// Average CPU usage by host over the last hour
timeseries avg_cpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
```

6. Summary and Next Steps

In this notebook you learned:

- The difference between dashboards and notebooks, and when to use each
- Dashboard architecture: tiles, sections, and variables
- Core design principles: clarity, audience awareness, actionability
- A structured workflow for creating dashboards
- Foundational DQL queries that appear on most dashboards

Next: In **DASH-02: Dashboard Hierarchy**, we explore the three-tier model — Executive, Operations, and Engineering dashboards — and how to design information density for each audience.
